CSE325 - OPERATING SYSTEMS

IELTS/GRE EXAM HALL MANAGEMENT SYSTEM

GROUP MEMBERS

Name	ID
Esrat Jahan Samiya	2023-2-60-002
Belal Hossain	2023-2-60-010
Md Nazmul Hossen	2023-2-60-012
Nusrat Jahan	2023-2-60-286

INTRODUCTION

This project simulates an exam hall management system specifically designed for IELTS and GRE examinations. Its primary purpose is to efficiently handle student registration, process exam applications, manage scheduling, and allocate rooms across multiple exam slots. The system includes essential features such as user authentication, where users can log in either as an admin or a student. Students can apply for exams, and their applications are subject to approval by the admin. The simulation employs multithreading to realistically model the behavior of students entering exam halls in two separate time slots, with room access controlled using semaphores and mutexes. Additionally, the system supports real-time status tracking, allowing both students and administrators to monitor application status, room assignments, and exam participation outcomes dynamically.

SYSTEM FEATURES OVERVIEW

The goal is to implement a concurrent and realistic simulation of an exam hall system using multithreading, semaphores, and mutexes to manage:

- Student applications and approval workflow
- Slot-based exam participation
- Safe and synchronized room assignment
- Accurate tracking of participation and rejection

DEMO WALKTHROUGH

Admin Flow:
Login → Approve students → Run simulation.

Student Flow:
Register → Apply → Check status.

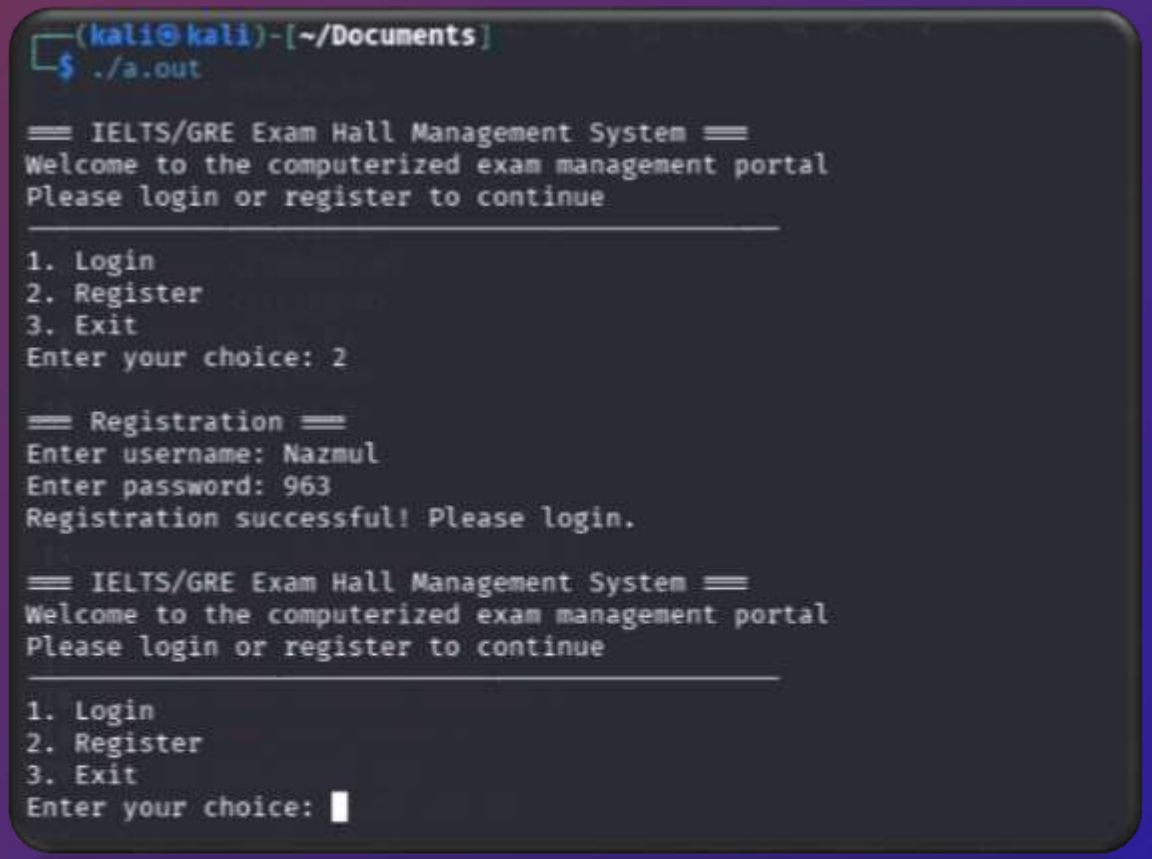
USER MANAGEMENT

Functions Used:

- *register_user():* Adds new students, checks for duplicate usernames.
- *login():* Verifies credentials, sets current_user.

Working Mechanism:

- *Admin:* Default account (admin:admin123).
- *Student:* Must register first.



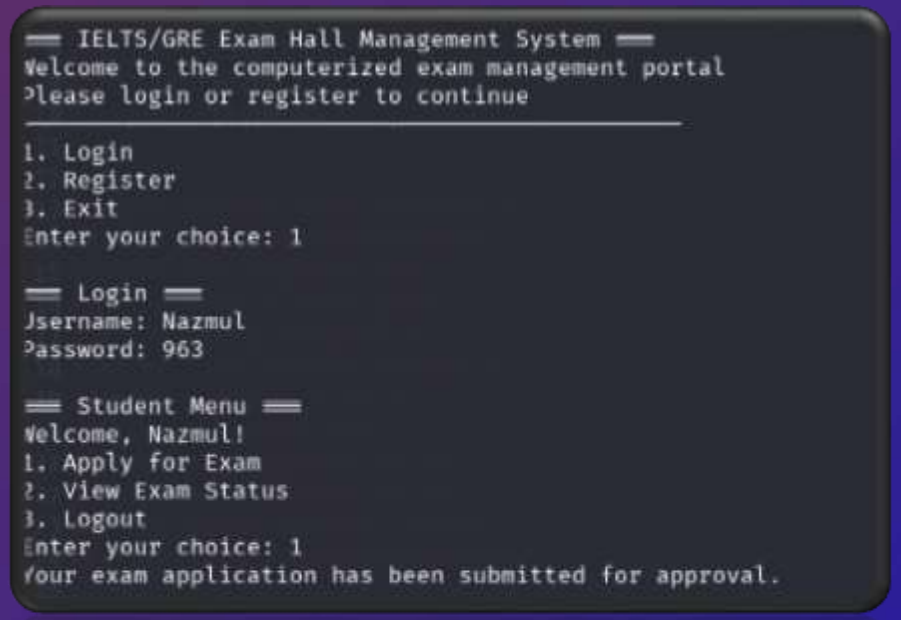
EXAM APPLICATION SYSTEM

Functions Used:

- *apply_for_exam():* Submits a student's exam request, sets status to PENDING.
- *approve_application():* Admin approves, student added to exam list.
- *reject_application():* Admin rejects application.

Workflow:

- Student applies → Admin reviews → Approved/Rejected.



APPROVAL POINT

In the admin panel, the approval point lets the admin manage student exam applications:

1. Manual Approval
Approve individual pending applications using the username.

2. Auto-Approve All
Approve all pending applications at once.

3. Reject Application
Reject any student's application by username.

4. View Applications
See all students with their application status.

```
== Admin Menu ==
1. Run Exam Simulation
2. View Student Status
3. Approve Student Manually
4. Auto-Approve All Pending Applications
5. View All Applications
6. Logout
Enter your choice: 4
Auto-approved 2 applications and added 2 new students to exam list.

== Admin Menu ==
1. Run Exam Simulation
2. View Student Status
3. Approve Student Manually
4. Auto-Approve All Pending Applications
5. View All Applications
6. Logout
Enter your choice:
```

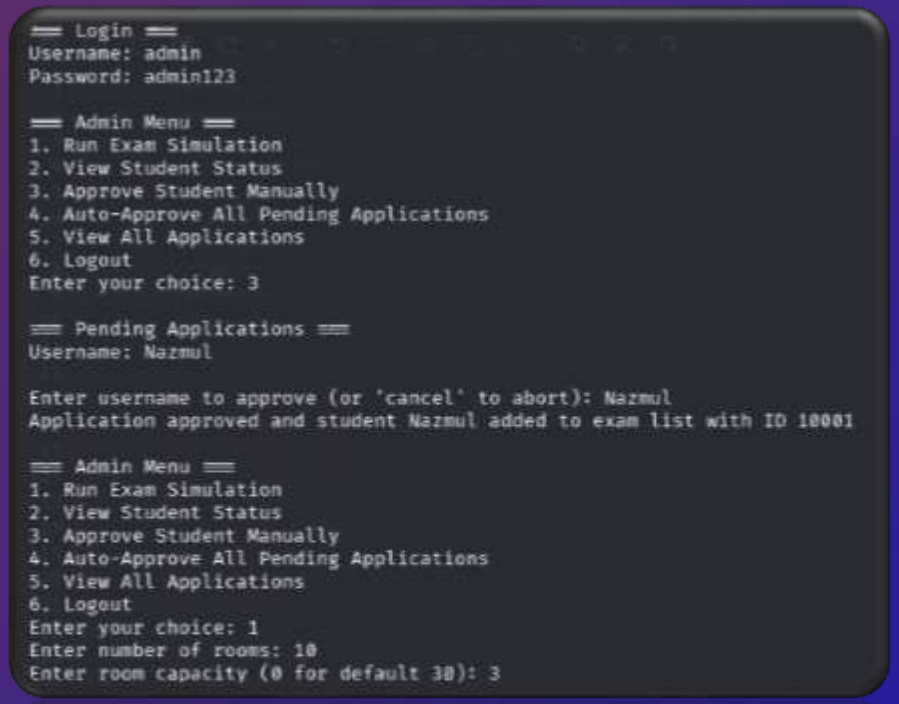
EXAM SIMULATION (CORE LOGIC)

Functions Used:

- *run_simulation():* Initializes rooms, threads, semaphores.
- *student_thread():* Tries to enter exam room, retries in second slot.
- *exam_supervisor_thread():* Controls exam start/end.

Key Mechanisms:

- *Semaphores:* Synchronize exam start/end.
- *Mutex Locks:* Ensure thread-safe room allocation.



```
== Login ==
Username: admin
Password: admin123

== Admin Menu ==
1. Run Exam Simulation
2. View Student Status
3. Approve Student Manually
4. Auto-Approve All Pending Applications
5. View All Applications
6. Logout
Enter your choice: 3

== Pending Applications ==
Username: Nazmul

Enter username to approve (or 'cancel' to abort): Nazmul
Application approved and student Nazmul added to exam list with ID 10001

== Admin Menu ==
1. Run Exam Simulation
2. View Student Status
3. Approve Student Manually
4. Auto-Approve All Pending Applications
5. View All Applications
6. Logout
Enter your choice: 1
Enter number of rooms: 10
Enter room capacity (0 for default 30): 3
```

ROOM ALLOCATION LOGIC

How It Works:

- ***First Slot:*** **Student tries to enter any room. If full → rejected for Slot 1.**
- ***Second Slot:*** **Re-attempt after room reset. If full → fully rejected.**

Functions Involved:

- ***try_lock():*** **Timeout-based mutex lock.**
- ***reset_rooms_for_second_slot():*** **Clears room occupancy.**

```
Enter your choice: 1
Enter number of rooms: 10
Enter room capacity (0 for default 30): 3

=== Exam Slot 1 is starting now at Wed May 21 17:01:14 2025

--- Slot 1 Room Assignments ---
Room 1: 0/3 students
Room 2: 0/3 students
Room 3: 0/3 students
Room 4: 0/3 students
Room 5: 0/3 students
Room 6: 0/3 students
Room 7: 0/3 students
Room 8: 0/3 students
Room 9: 0/3 students
Room 10: 0/3 students
-------------------------

Student 10001 (Nazmul) entered Room 1 in first slot

=== Exam Slot 1 has ended at Wed May 21 17:01:17 2025
Student 10001 (Nazmul) left Room 1 after first slot exam

=== Exam Slot 2 is starting now at Wed May 21 17:01:17 2025

--- Slot 2 Room Assignments ---
Room 1: 0/3 students
Room 2: 0/3 students
Room 3: 0/3 students
Room 4: 0/3 students
Room 5: 0/3 students
Room 6: 0/3 students
Room 7: 0/3 students
Room 8: 0/3 students
Room 9: 0/3 students
Room 10: 0/3 students
```

STATUS TRACKING & REPORTING

Functions Used:

- ***view_exam_status():*** **Student application and exam info.**
- ***view_student_status():*** **Admin sees all student data.**
- ***print_summary(): Shows exam statistics.***

Output Example:

- **Student 10001 (Alice):**
- **Slot: 1**
- **Room: 3**

```
=== Exam Attendance Summary ===
Total students registered: 1

Students who entered first slot exam: 1
Students who entered second slot exam: 0
Students who couldn't enter either slot: 0

First slot room-wise attendance:
Room 1: 1/3 students
Room 2: 0/3 students
Room 3: 0/3 students
Room 4: 0/3 students
Room 5: 0/3 students
Room 6: 0/3 students
Room 7: 0/3 students
Room 8: 0/3 students
Room 9: 0/3 students
Room 10: 0/3 students

Second slot room-wise attendance:
Room 1: 0/3 students
Room 2: 0/3 students
Room 3: 0/3 students
Room 4: 0/3 students
Room 5: 0/3 students
Room 6: 0/3 students
Room 7: 0/3 students
Room 8: 0/3 students
Room 9: 0/3 students
Room 10: 0/3 students

Exam duration per slot: 3 seconds (simulated)
```

THREADING & SYNCHRONIZATION

We uses multi-threading and synchronization mechanisms

1. Threading Model

Student Threads (student_thread)

- Each student is represented by a separate thread.
- Students attempt to enter Slot 1 first. If they fail (due to room capacity or timeout), they retry in Slot 2.
- Threads synchronize using semaphores (exam_start_sem, exam_end_sem) to start/end exams.

Supervisor Threads (exam_supervisor_thread)

- One supervisor thread per exam slot.
- Signals the start/end of exams using semaphores.

THREADING & SYNCHRONIZATION

Synchronization Tools:

- Semaphores (exam_start_sem, exam_end_sem):

Help start and end the exam at the right time for all students.

- Mutex Locks (entry_lock, hall_lock):

Make sure only one student enters or exits at a time to avoid problems.

Entry Locks (entry_lock[slot]):

- Controls access to the exam slot entry process.

ERROR HANDLING & CLEANUP

Key Functions:

- try_lock() – Tries to lock safely; avoids system getting stuck (deadlock).
- cleanup_resources() – Cleans up memory and closes all locks when done.

Safety Checks:

- Checks if memory is available before using it.
- Checks if threads start properly; shows error if not.

CONCLUSION

We built a working exam hall system for IELTS and GRE that includes secure login, different access for students and admins, and an approval system for exams. Using multithreading, we made it act like real students joining exams at the same time and handled room management well. The system tracks everything in real time using locks and semaphores. In the future, we can add a graphical interface and a database to make it easier to use and more powerful.